© 2025 IEEE – Accepted to IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) 2025

arXiv:2503.12204v4 [cs.RO] 1 Sep 2026

D4orm: Multi-Robot Trajectories with Dynamics-aware Diffusion Denoised Deformations

Yuhao Zhang, Keisuke Okumura, Heedo Woo, Ajay Shankar, Amanda Prorok

***Abstract*— This work presents an optimization method for generating kinodynamically feasible and collision-free multi-robot trajectories that exploits an incremental denoising scheme in diffusion models. Our key insight is that high-quality trajectories can be discovered merely by denoising noisy trajectories sampled from a distribution. This approach has no learning component, relying instead on only two ingredients: a dynamical model of the robots to obtain feasible trajectories via rollout, and a fitness function to guide denoising with Monte Carlo gradient approximation. The proposed framework iteratively optimizes a deformation for the previous trajectory with the current denoising process, allows *anytime* refinement as time permits, supports different dynamics, and benefits from GPU acceleration. Our evaluations for differential-drive and holonomic teams with up to 16 robots in 2D and 3D worlds show its ability to discover high-quality solutions faster than other black-box optimization methods such as MPPI. In a 2D holonomic case with 16 robots, it is almost twice as fast. As evidence for feasibility, we demonstrate zero-shot deployment of the planned trajectories on eight multirotors.**

I. Introduction

Generating conflict-free state-to-state trajectories for robot teams is a critical task when operating multiple robots in a shared workspace, and is frequently required in areas such as warehouse automation [1] and transportation systems [2]. In such scenarios, robots need to operate with tight coordination while minimizing redundant movements to optimize various metrics of system performance (such as flowtime). This is achieved by optimizing trajectory plans for the entire team.

While the quality of collision-free trajectories is easily specified, the corresponding "joint" optimization problem, where all robots' states are considered jointly, is often unsuitable for classical numerical methods. This is primarily due to the size of the joint configuration space, which grows exponentially with the number of robots [3]. Furthermore, the optimization landscape is generally non-convex, can contain multiple equivalent solutions, and has constraints that make it difficult to compute exact analytical gradients. Recent trends therefore relax the objective of synthesizing globally optimal trajectories by decoupling robot-wise states from the joint representation, while often choosing suboptimal and conservative actions due to an incomplete state representation [4]–[8]. Although their advances are remarkable and their decen-

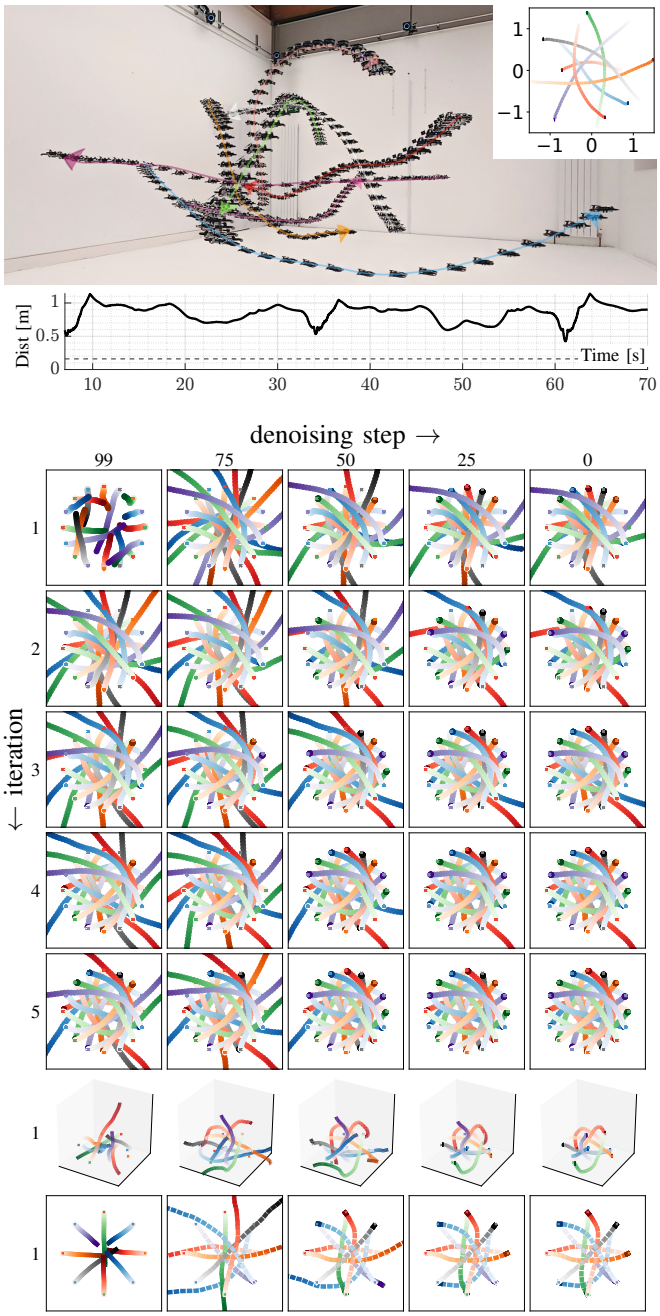


Fig. 1: Zero-shot deployment of deconflicted trajectories with eight multirotors (top) with denoised trajectories (top-right inset); the minimum pairwise inter-robot distance shows safety margin w.r.t robot radius (solid and dotted black lines). The lower plots visualize denoising of 16 2D-holonomic, 8 3D-holonomic, and 8 differential-drive robots. Each plot shows the result of adding deformation denoised at current step to trajectory given by previous iteration.

The authors are with the University of Cambridge, UK. KO is also with National Institute of Advanced Industrial Science and Technology (AIST), Japan. Emails: {yz981,ko393,hw527,as3233,asp45}@cst.cam.ac.uk.

This research was funded in part by the EPSRC funded INFORMED-AI project EP/Y028732/1 and in part by European Research Council (ERC) Project 949940 (gAIa). We gratefully acknowledge their support. KO was partially supported by JSPS Overseas Research Fellowship.

Code and video: https://github.com/proroklab/d4orm

tralization possibilities appealing, the lack of coordination guarantees makes them unsuitable for safety-critical multi-robot platforms, which could be the future infrastructure that underpins our daily lives. With these in mind, reliable and scalable multi-robot trajectory optimization methods in the joint representation remain a pivotal technology.

In this work, we investigate a novel optimization mechanism that generates collision-free and kinodynamically feasible trajectories in the joint representation. We sidestep each of the aforementioned limitations of classical numerical methods by employing strategies from diffusion models, which have successfully captured high-dimensional features in various domains, originally in image generation [9], [10], but also in robot motion generation [11]. Specifically, building on *model-based diffusion* [12], we propose D4ORM ("deform"), that iteratively denoises joint control-space trajectories for the entire team by using diffusion denoising as a black-box (gradient-free) optimization tool. Each denoising step refines a *deformation* from the previous step's solution, and is guided by an on-the-fly Monte Carlo score approximation combined with an interpretable fitness function. Unlike the traditional use of diffusion models, this dynamics-driven approach does not require supervised data and therefore support different robot dynamics. The proposed framework is an *anytime* algorithm, and benefits from GPU parallelization of Monte Carlo rollouts.

Illustrated as snapshots in Fig. 1, our key insight is that, starting from noisy/infeasible trajectories, high-quality trajectories are discovered entirely through denoising, a process that only needs a model of the robot dynamics and a fitness (reward) function. We present a range of studies in terms of computation time, success rate, and solution quality, with teams of 8-16 robots navigating in a dense obstacle-free workspace towards antipodal points on a circle, and with three distinct dynamics. Our method retrieves deconflicted trajectories in these high-dimensional, potentially non-convex solution spaces within reasonable deadlines while outperforming baselines. For instance, we show planning for 16 2D-holonomic robots in approx. $2\,\mathrm{s}$ on average over a planning horizon of 100 steps, which is near $2\times$ faster than other sampling-based optimization methods. Finally, we deploy a team of eight multirotors in a zero-shot manner, proving the kinodynamic feasibility of generated trajectories.

II. Related Work

There exists a rich literature on multi-agent and multi-robot trajectory optimization. Instead of decoupled approaches that perform an optimization process locally per-robot, e.g., [4]–[8], we are interested in improving the performance of coupled approaches that obtain coordinated trajectories. These methods are often categorized as either *sampling-based motion planning (SBMP)* or *numerical optimization*. The former, SBMP, uses discrete combinatorial search over a roadmap approximation of the joint configuration space, constructed by random sampling [13]–[15]. The latter reduces deconfliction to a numerical optimization formulation and then typically applies well-established solvers to obtain a solution [16]–[18]. Both categories suffer from the curse of dimensionality as the number of agents increases.

Our proposed denoising method belongs to the orthogonal category of what we call *sampling-based optimization* methods, which, unlike SBMP, aim to sample a solution trajectory from a high-dimensional space, by massive sampling attempts. Unlike numerical optimization and SBMP, sampling-based optimization methods have an easily accelerated structure by parallelizing this trajectory collection with GPUs. While such methods have been successfully applied to single-robot control in challenging environments [19], applying it to multi-robot systems also faces the same dimensionality issue. In fact, many existing sampling-based optimization methods for multi-robot control use decoupled representations [20]–[22]. In contrast, we aim to sample the joint trajectories directly through diffusion denoising, a process more equipped to handle high dimensionality.

Diffusion models, which originally received considerable attention in image generation [9], [10], are a popular choice for learning-based robot motion generation [11], [23]. It is chiefly characterized by its ability to extract features in very high-dimensional spaces, such as those that represent an image, or multi-robot motion. Indeed, diffusion has been applied in multi-agent use-cases such as in trajectory prediction [24], or motion planning combined with constrained optimization [25], [26]. While such diffusion-based generative methods aim to imitate demonstration trajectories, recent work has also proposed learning-free trajectory generation via Monte Carlo gradient approximation, called model-based diffusion (MBD) [12]. Our work builds on MBD, explained in detail in Sec. IV, which was originally designed for single-robot planning. We expand it to multi-robot settings with the introduction of team-level fitness (reward) functions plus a key methodological innovation of iteratively optimizing *deformation* vectors to solve more challenging problems.

III. Problem Definition

In the following, we use a braced superscript $\{k\}$ to denote robot indices, and a plain subscript i for a denoising step.

A target system consists of a team of n homogeneous spherical robots $R = \{1, 2, \dots n\}$ each with a radius R_a. Their dynamics are each governed by $\dot{x} = f_{\mathrm{dyn}}(x, u)$, where $x \in \mathcal{X} \subset \mathbb{R}^{d_x}$ and $u \in \mathcal{U} \subset \mathbb{R}^{d_u}$ denote state and control vectors, respectively. Given their joint initial and terminal states, $\mathcal{S}, \mathcal{T} \in \mathcal{X}^n$, the **objective** is to generate a list, τ, representing a set of n collision-free and kinodynamically feasible trajectories. Specifically, for robots $k, l \in R$, $\tau^{\{k\}} \in (\mathcal{X} \times \mathcal{U})^H$ represents the trajectory for the k-th robot over some finite horizon H as a sequence of state-control pairs, sampled at time intervals $\triangle t \in \mathbb{R}_{>0}$, which satisfies:

$$
\begin{aligned}
\tau^{\{k\}}[t+1] &= \texttt{RK4}(\tau^{\{k\}}[t], f_{\mathrm{dyn}}, \triangle t) && \text{(feasibility)} \\
\tau^{\{k\}}[1].x &= \mathcal{S}^{\{k\}} && \text{(init. cond.)} \\
\tau^{\{k\}}[H].x &= \mathcal{T}^{\{k\}} && \text{(term. cond.)} \\
\mathrm{Dist}(\tau^{\{k\}}[t], \tau^{\{l\}}[t]) &> 2 \cdot \mathrm{R}_a && \text{(safety)} \qquad (1)
\end{aligned}
$$

where RK4 denotes the fourth-order Runge-Kutta integration of system dynamics, and $\text{Dist}(\cdot)$ denotes the Euclidean distance between the position components of two trajectory states. The quality, i.e., the reward to be maximized, of a feasible solution is evaluated in relation to the total travel time, represented in the following form:

$$\frac{1}{nH}\sum_{k\in R}\sum_{t=0}^{H}\text{Ind}\left[\tau^{\{k\}}[t].x = \mathcal{T}^{\{k\}}\right] \tag{2}$$

where $\text{Ind}[\cdot] = 1$ if the condition is true; zero otherwise.

IV. Deconflicting with Diffusion Denoising

We now describe the denoising optimization process that obtains a solution to the problem defined in Eq. (1). While finding an optimal solution is non-trivial due to the high dimensionality of the space, it is relatively straightforward to evaluate candidate solutions based on a fitness (reward) function. Our approach, therefore, is to use rewards observed in a batch of candidate rollouts generated using a given f_{dyn} to iteratively refine new candidates. We begin by explicitly defining such a reward function that represents Eq. (2) for multi-robot navigation scenarios. We use the term 'reward' to be consistent with *model-based diffusion* [12].

A. Reward Structure For Multi-Robot Trajectories

We define the general multi-robot reward as comprised of two parts: a quality of goal navigation reward, r_{goal}, and a reward for inter-robot safety, r_{safe}. Using a weighting parameter $w_t \in \mathbb{R}$, our reward for the joint trajectory τ over the horizon H is expressed as

$$r(\tau) = \frac{1}{nH}\sum_{t=1}^{H}\sum_{k\in R}\Big(r_{\text{goal}}(\tau^{\{k\}},t) + w_t \cdot r_{\text{safe}}(\tau^{\{k\}},t)\Big). \tag{3}$$

The first term optimizes the k-th trajectory $\tau^{\{k\}}$ based on the objective function in Eq. (2), but with a dense reward structure, while the second term explicitly penalizes collisions. Specifically, in our setting, given a target position $p_{\mathcal{T}}^{\{k\}} \in \mathcal{T}^{\{k\}}$,

$$r_{\text{goal}}(\tau^{\{k\}},t) = 1 - \frac{\|p^{\{k\}}[t] - p_{\mathcal{T}}^{\{k\}}\|}{\|p^{\{k\}}[1] - p_{\mathcal{T}}^{\{k\}}\|} \tag{4}$$

$$r_{\text{safe}}(\tau^{\{k\}},t) = \begin{cases} -1, & \text{if } \|p^{\{k\}}[t] - p^{\{l\}}[t]\| \le 2\text{R}_a + \epsilon \\ 0, & \text{otherwise} \end{cases} \tag{5}$$

where $\epsilon \in \mathbb{R}_+$ defines a safety margin, and $l \in R \setminus k$.

The simple reward design from Eq. (3) guides the denoising process which we describe in the following subsections.

B. Diffusion Denoising without Data

We begin by providing a general overview of diffusion and denoising, adapted here for completeness from prior work on model-based diffusion [12]. For now, consider a single-robot scenario, i.e., τ as a *single robot* trajectory $\tau \in (\mathcal{X}\times\mathcal{U})^H$. In the scheme of diffusion models, we are interested in sampling a solution from a target distribution p_0 that assigns a high density to a solution with higher rewards. Using a temparture parameter $\lambda \in \mathbb{R}_{>0}$, we represent p_0 as:

$$p_0(\tau) \propto \exp\left(\frac{r(\tau)}{\lambda}\right) \tag{6}$$

Sampling a solution directly from p_0 is, however, significantly challenging because τ lies in a high-dimensional space, e.g., $H(d_x + d_u)$ for the single-robot case. Diffusion models are a successful framework for dealing with this problem through an iterative denoising process. The forward process slowly corrupts the structure of the data by adding noise. Mathematically, each sample τ_i is corrupted sequentially from p_0 to an isotropic Gaussian p_N with schedule parameters α_t, using Gaussian noise of

$$p_{i|i-1}(\tau_t|\tau_{i-1}) = \mathcal{N}\left(\sqrt{\alpha_i}\tau_{i-1}, (1-\alpha_i)I\right) \tag{7}$$

The backward process $p_{i-1|i}(\cdot)$ is the reverse of the forward process $p_{i|i-1}(\cdot)$. When this backward distribution is available, we could reconstruct p_0 through:

$$p_{i-1}(\tau_{i-1}) = \int p_{i-1|i}(\tau_{i-1}|\tau_i)p_i(\tau_i)d\tau_i \tag{8}$$

$$p_0(\tau_0) = \int p_N(\tau_N)\prod_{i=N}^{1} p_{i-1|i}(\tau_{i-1}|\tau_i)d\tau_{1:N} \tag{9}$$

Traditional diffusion models solve the backward process by learning a score function from the data. Meanwhile, the model-based diffusion (MBD) [12] employs Monte Carlo score estimation. Using the inverse scale of the forward process, MBD employs the denoising process with

$$\tau_{i-1} = \frac{1}{\sqrt{\alpha_i}}\left(\tau_i + (1-\bar{\alpha}_i)\nabla_{\tau_i}\log p_i(\tau_i)\right) \tag{10}$$

where $\bar{\alpha}_i = \prod_{j=1}^{i}\alpha_j$. The score term is approximated by

$$\nabla_{\tau_i}\log p_i(\tau_i) \approx -\frac{\tau_i}{1-\bar{\alpha}_i} + \frac{\sqrt{\bar{\alpha}_i}}{1-\bar{\alpha}_i}\bar{\tau} \tag{11}$$

Here $\bar{\tau}$ is a weighted average of samples around τ_i, based on the target distribution p_0:

$$\Gamma \sim \mathcal{N}\left(\frac{\tau_i}{\sqrt{\bar{\alpha}_i}}, \left(\frac{1}{\bar{\alpha}_i} - 1\right)I\right),\ \bar{\tau} = \frac{\sum_{\tau\in\Gamma} p_0(\tau)\tau}{\sum_{\tau\in\Gamma} p_0(\tau)} \tag{12}$$

Taking Eq. (10) to (12) together, the one-step denoising in MBD is simply summarised as:

$$\tau_{i-1} = \sqrt{\bar{\alpha}_{i-1}}\cdot\bar{\tau} \tag{13}$$

In contrast to other sampling-based methods like MPPI [19], which performs a softmax update, MBD introduces an extra noise schedule to both the sampling and update phases.

C. Denoising for Trajectory Optimization: D4ORM Basics

Now that we have established the process for denoising, the remainder of this section describes how we apply it for optimization. Keeping in line with the diffusion terminology, we will slightly abuse the term 'noisy trajectory' to mean a trajectory of states/controls sampled from a distribution other

Algorithm 1 Denoising Optimization Process

1: $U_N \sim \mathcal{N}(\mathbf{0}, I)$ ▷ *initialize control trajectory*
2: **for** $i \leftarrow N$ to 1 **do**
3: Sample M control trajectories:

$$\Gamma_u \sim \mathcal{N}\left(\frac{U_i}{\sqrt{\bar{\alpha}_i}}, \left(\frac{1}{\bar{\alpha}_i} - 1\right) I\right)$$

4: Sample M trajectories: $\Gamma \leftarrow \texttt{rollout}\,(\mathcal{S}, \Gamma_u)$
5: Compute Monte Carlo estimation:

$$\bar{U} \leftarrow \frac{\sum_{\tau\in\Gamma} p_0(\tau)(\tau.u)}{\sum_{\tau\in\Gamma} p_0(\tau)},\ p_0(\tau) \approx \exp\left(\frac{r(\tau)}{\lambda}\right)$$

6: Perform one-step denoising: $U_{i-1} \leftarrow \sqrt{\bar{\alpha}_{i-1}}\,\bar{U}$
7: **return** $\texttt{rollout}\,(\mathcal{S}, U_0)$

than the target distribution. We note that the kinodynamic feasibility condition forces states and controls to be consistent using a given f_{dyn}. Thus, sampling them independently is impractical since the distribution approaches a Dirac delta function. Therefore, similar to MBD, we sample control trajectories instead, and then retrieve state-control trajectories with rollout from the initial state, i.e, iteratively generate a state sequence with $x[t+1] = \texttt{RK4}(x[t], u[t], f_{\text{dyn}}, \triangle t)$ given a control sequence. This process on the trajectory batch Γ can be effectively parallelized using GPUs.

Algorithm 1 describes how MBD actually solves the trajectory optimization. Starting from a noisy control trajectory (Line 1), $U_N \in \mathcal{U}^H$ for single-robot, MBD first samples M control trajectories surrounding the previous iteration (Line 3). These controls are then converted into feasible trajectories (Line 4) and used to approximate the score term (Line 5), followed by an update from Eq. (13) (Line 6). The denoised variable constitutes a solution (Line 7). In practice, rewards are normalized within a batch to stabilize the denoising process.

MBD was originally developed for single-robot trajectory optimization. Inspired by its ability to synthesize trajectories in high-dimensional space, the remaining part leverages it for multi-robot trajectory deconfliction. We first extend the control trajectory, assumed in the previous description to be $U_i \in \mathcal{U}^H$, to the joint control trajectory for all robots, i.e. $U_i \in \mathcal{U}^{nH}$. The rollout at Line 4 is then performed for n robots, and produces a batch of joint trajectories, each constituting n robot trajectories $\tau \in (\mathcal{X} \times \mathcal{U})^{nH}$. The rest of the denoising process operates in this joint trajectory representation to derive a solution τ to the multi-robot trajectory formulation in Sec. III.

D. Iterative Optimization of Deformations: D4ORM *Core*

Since finding solutions to multi-robot deconfliction has varying complexity depending on the configuration, using Alg. 1 directly is insufficient, as it requires a predefined number of denoising steps N and lacks the anytime property. Therefore, we propose performing the denoising process iteratively, using Alg. 1 with a fixed small N several times.

As shown in Alg. 2, the crux here is that instead of simply using MBD to synthesize control trajectories from scratch, we synthesize a deformation control vector $\Delta U \in \mathcal{U}^{nH}$ for the previous iteration's trajectory. In other words, we optimize ΔU via denoising to update the control trajectory in a solution τ as $\tau.u \leftarrow \tau.u + \Delta U$. The samples in Alg. 1 now become deformation vectors rather than control trajectories, and the reward for a sampled deformation ΔU_{sample} is computed by rolling out $\tau.u + \Delta U_{\text{sample}}$, where τ is the trajectory obtained from the previous iteration and is fixed to serve as the base trajectory when denoising ΔU at current iteration. The rationale is that each denoising process guides τ towards the target distribution p_0 and thus the amount of deformation becomes smaller and smaller over iterations. This makes it an *anytime* planning algorithm; over time, we can expect the solutions to get better and better. The decision of when to stop depends on the applications and user requirements. Some may stop refining when a feasible solution is reached, others may stop at the planning deadline.

Algorithm 2 Iterative Denoising

1: Initialize τ
2: **while** not interrupted **do**
3: Get a deformation control vector ΔU using Alg. 1 and τ
4: $\tau \leftarrow \texttt{rollout}\,(\mathcal{S}, \tau.u + \Delta U)$
5: **return** τ

The validity of this iterative denoising comes from MBD with the reverse process $p_{i-1|i}(\cdot)$ approximated by on-the-fly Monte Carlo score estimation. Traditional diffusion models with neural networks that learn from data cannot cope with the shift in the target distribution of deformations during successive iterations. Further, we observe empirically that initializing ΔU_N with a sample from a standard Gaussian distribution does not produce a steady refinement over the iterations, and instead, our implementation uses a zero vector as ΔU_N. Our hypothesis is that introducing a zero-mean inductive bias *helps* the denoising process approximate the optimal deformation quicker, while adding large deformations to the control trajectory is more likely to cause dramatic changes to the position trajectory.

V. EVALUATIONS

We evaluate the performance of our trajectory optimization method qualitatively and quantitatively through a variety of metrics. A key feature of our method is the ability to use a variety of kinodynamic models, and thus we show evaluations on three types of systems:

- **Differential Drive** mixed-order integrator system that represents wheeled robots with state $x = [p_x, p_y, \theta, v]^\top$, control $u = [\omega, a]^\top$, and $f_{\text{dyn}}(x, u) = [v\cos\theta, v\sin\theta, \omega, a]^T$;
- **2D Holonomic** double-integrator system that represents ground robots with state $x = [p_x, p_y, v_x, v_y]^\top$, control $u = [a_x, a_y]^\top$, and $f_{\text{dyn}}(x, u) = [v_x, v_y, a_x, a_y]^T$;
- **3D Holonomic** double-integrator system, which is a 3D version of the 2D case.

All evaluations are carried out on a laptop PC with an Intel Core i9-13900HX CPU, equipped with an NVIDIA RTX

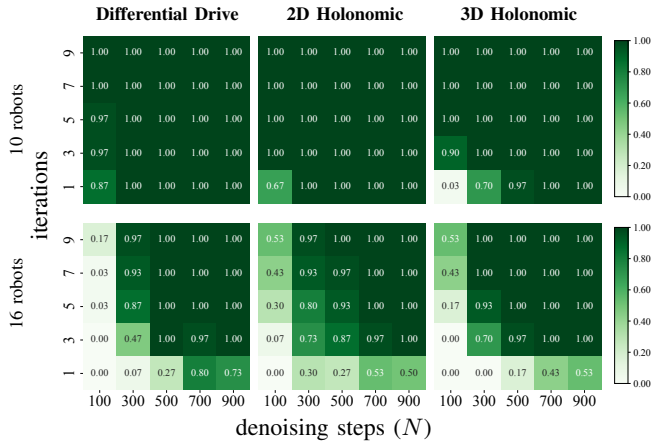


Fig. 2: Sensitivity of the success rate to the number of diffusion steps and deformation iterations.

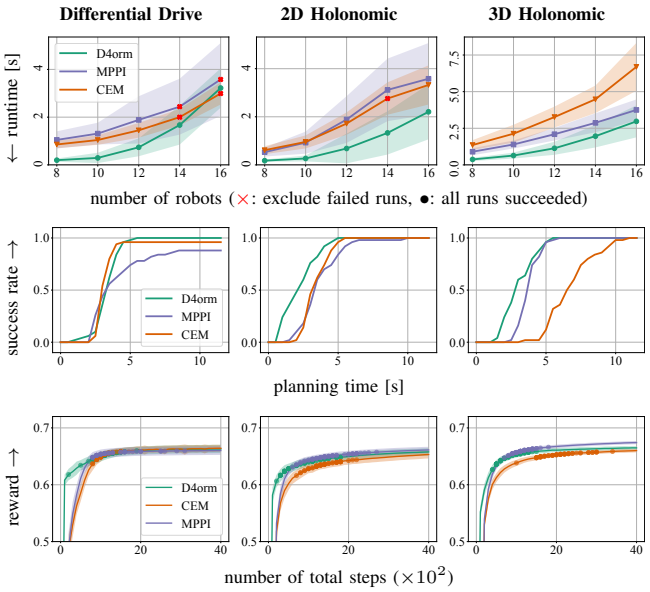


Fig. 3: Empirical results against baseline methods. The upper figures show the time required by each method to find initial feasible solutions over different numbers of agents. The middle ones show the planner's ability to find a solution in the given time, focusing on the case of 16 robots. The bottom ones show how quickly each method finds plausible solutions, along with the total number of steps (iters $\times N$ for D4ORM), using 16 robots each. Instance-wise, when to find the initial solution is marked with •.

4080 GPU. Our implementation is based on [12], coded in Python with `JAX` [27] library to streamline GPU acceleration. We use planning horizon $H = 100$ with $\Delta t = 0.1$ and $M = 2048$ trajectory rollouts to approximate gradients (scores).

A. Qualitative Analysis

Figure 1 depicts a qualitative investigation into the intermediate steps of the denoising process for each of these systems. The problem considers a navigation task involving 8 and 16 robots placed on a circle (sphere, in 3D) with their respective destinations placed at antipodal locations. Such scenarios have served as typical benchmarks in the multi-robot planning literature, as they always force planners to make non-trivial deconfliction efforts. The snapshots in Fig. 1 depict the refinement across different denoising steps, i.e., over i in Alg. 1, and iterations of Alg. 2. The figure illustrates that the trajectories converge to their targets and are collision-free as the denoised steps increase, despite its enormous number of optimization variables, i.e., $d_u \times H \times n$. As seen in the 2D holonomic case, which has 3200 variables, iterative deformation optimization then compensates for incomplete trajectories and further improves solution quality by escaping local minima, by resetting the diffusion noise scheduler.

B. Quantitative Evaluations: Sensitivity

We now evaluate the performance of our iterative denoising process in terms of its success rate over the number of iterations and the number of denoising steps. A solution is "successful" when conditions in Eq. (1) are met, i.e., feasible, conflict-free trajectories are found for all robots. This evaluation is crucial from a practical perspective, since the total number of rollout operations, which directly affects the wall time, remains the same for $3\,\text{iters} \times 100\,\text{steps}$ and $1\,\text{iters} \times 300\,\text{steps}$, even though these can produce some differences based on the difficulty of the problem.

Figure 2 presents an overview of success rates for 10 and 16 robot problems with the three dynamics models, averaged over 30 runs with different initial seeds. We observe that several of the 10-robot cases are solved even with a single denoising iteration. For 16 robots, we see some variability, and the results indicate that either increasing the denoising steps (N) or the number of iterations improves the success rate. Nevertheless, in the following quantitative evaluations, we set $N = 100$ to provide a more general setup while comparing against baselines. This choice keeps the computational time per iteration low, enabling more flexible termination of the algorithm.

C. Quantitative Evaluations: Baselines

Next, we contrast our method against two highly effective and competitive baseline methods that belong to the sampling-based optimization category: model-predictive path integral (MPPI) [19] and Cross Entropy Method (CEM) [28]. For evaluation, these methods are adapted to perform sampling in the joint control space. We evaluate three key metrics of interest: *runtime*, to quantify scalability against varying team sizes by measuring clock time for computing successful solutions, *reward*, to quantify solution quality according to Eq. (2) when planning is allowed to continue for longer, and, *success rate*, as defined previously, but measured against a given planning deadline.

Figure 3 shows a complete overview of our comparisons over 50 runs. We observe a noticeable improvement in **runtime** and success rate for our method against *all* baselines for *all* robot models and for *almost all* team sizes. In particular, the **success rate** plots (middle row) show that for the 2D holonomic case, our approach can always retrieve kinodynamically feasible and collision-free trajectories for

16-robots teams given a $5\,\mathrm{s}$ deadline. The average runtime is lower, $\approx 2\,\mathrm{s}$, which is a near $2\times$ improvement over MPPI and CEM. The **reward** plots at the bottom show the 'anytime planning' nature using the number of steps, which serves as a metric for assessing runtime independently of computing environments. Recall that the reward is an indicator for solution quality, and as such, all methods converge to similar reward values given sufficient time for refinement. However, our proposed method shows two key advancements. First, its reward curves show a steeper gradient during early steps, indicating that we obtain high-quality solutions faster. Second, as indicated by the solid dots on the curves, our high-quality solutions are successful (i.e., conflict-free) earlier than other methods of similar quality. This is most noticeable in the 3D holonomic case, where our method offers a dramatic $\approx 60\,\%$ reduction in the total steps required compared to CEM. These improvements are owing to the ability of diffusion denoising to capture complicated, multimodal reward distributions in high-dimensional spaces.

D. Real-World Deployment

As evidence of safety and feasibility, we deploy trajectories generated by denoising in a zero-shot manner for a team of eight multirotors [29]. Fig. 1 presents the deployment snapshot as well as the minimum inter-robot distance measured over several repeated executions beyond one minute, demonstrating that the robots maintain a safe distance. The video is available in the supplementary materials, which also include demonstrations of more advanced planning scenarios with various obstacle densities, a larger number of robots, and random target assignments.

VI. Discussion

We studied a multi-robot trajectory optimization framework with diffusion denoising. Given robot dynamics and a reward function, the framework iteratively applies a deformation vector to the current candidate solution, which is guided by a Monte Carlo score approximation. Despite the high dimensionality of deconfliction problems, our evaluations demonstrate that this simple process successfully retrieves collision-free trajectories for the entire team within reasonable deadlines. There are technical considerations surrounding the framework's reliance on numerous and long-horizon rollouts, which is currently the most expensive subprocess. This necessitates using GPUs to speed up gradient approximation. The framework can be applied to non-spherical shapes at the cost of more expensive inter-robot collision checking. Generally, improving sampling efficiency is thus important. Nevertheless, our future work is addressing several directions unexplored in this paper. These include handling obstacle-rich settings, and heterogeneous teams.

References

[1] P. R. Wurman, R. D'Andrea, and M. Mountz, "Coordinating hundreds of cooperative, autonomous vehicles in warehouses," *AI Mag.*, 2008.

[2] K. Dresner and P. Stone, "A multiagent approach to autonomous intersection management," *J. Artif. Intell. Res.*, 2008.

[3] J. E. Hopcroft, J. T. Schwartz, and M. Sharir, "On the complexity of motion planning for multiple independent objects; pspace-hardness of the "warehouseman's problem"," *Int. J. Robot. Res.*, 1984.

[4] Y. Chen, M. Cutler, and J. P. How, "Decoupled multiagent path planning via incremental sequential convex programming," in *ICRA*, 2015.

[5] C. E. Luis and A. P. Schoellig, "Trajectory generation for multiagent point-to-point transitions via distributed model predictive control," *IEEE Robot. Autom. Lett.*, 2019.

[6] J. Tordesillas and J. P. How, "MADER: Trajectory planner in multiagent and dynamic environments," *IEEE Trans. Robot.*, 2021.

[7] X. Zhou, J. Zhu, H. Zhou, C. Xu, and F. Gao, "EGO-Swarm: A fully autonomous and decentralized quadrotor swarm system in cluttered environments," in *ICRA*, 2021.

[8] B. Şenbaşlar, W. Hönig, and N. Ayanian, "RLSS: Real-time, decentralized, cooperative, networkless multi-robot trajectory planning using linear spatial separations," *Auton. Robots*, 2023.

[9] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *NeurIPS*, 2020.

[10] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," in *CVPR*, 2022.

[11] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *Int. J. Robot. Res.*, 2023.

[12] C. Pan, Z. Yi, G. Shi, and G. Qu, "Model-based diffusion for trajectory optimization," in *NeurIPS*, 2024.

[13] P. Švestka and M. H. Overmars, "Coordinated path planning for multiple robots," *Robot. Auton. Syst.*, 1998.

[14] K. Solovey, O. Salzman, and D. Halperin, "Finding a needle in an exponential haystack: Discrete RRT for exploration of implicit roadmaps in multi-robot motion planning," *Int. J. Robot. Res.*, 2016.

[15] K. Okumura and X. Défago, "Quick multi-robot motion planning by combining sampling and search," in *IJCAI*, 2023.

[16] F. Augugliaro, A. P. Schoellig, and R. D'Andrea, "Generation of collision-free trajectories for a quadrocopter fleet: A sequential convex programming approach," in *IROS*, 2012.

[17] A. Kushleyev, D. Mellinger, C. Powers, and V. Kumar, "Towards a swarm of agile micro quadrotors," *Auton. Robots*, 2013.

[18] V. K. Adajania, S. Zhou, A. K. Singh, and A. P. Schoellig, "AMSwarm: An alternating minimization approach for safe motion planning of quadrotor swarms in cluttered environments," in *ICRA*, 2023.

[19] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Information-theoretic model predictive control: Theory and applications to autonomous driving," *IEEE Trans. Robot.*, 2018.

[20] L. Streichenberg, E. Trevisan, J. J. Chung, R. Siegwart, and J. Alonso-Mora, "Multi-agent path integral control for interaction-aware motion planning in urban canals," in *ICRA*, 2023.

[21] E. Trevisan and J. Alonso-Mora, "Biased-MPPI: Informing sampling-based model predictive control by fusing ancillary controllers," *IEEE Robot. Autom. Lett.*, 2024.

[22] C. Jiang, "Distributed sampling-based model predictive control via belief propagation for multi-robot formation navigation," *IEEE Robot. Autom. Lett.*, 2024.

[23] M. Janner, Y. Du, J. B. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," in *ICML*, 2022.

[24] C. Jiang, A. Cornman, C. Park, B. Sapp, Y. Zhou, D. Anguelov, *et al.*, "MotionDiffuser: Controllable multi-agent motion prediction using diffusion," in *CVPR*, 2023.

[25] Y. Shaoul, I. Mishani, S. Vats, J. Li, and M. Likhachev, "Multi-robot motion planning with diffusion models," in *ICLR*, 2025.

[26] J. Liang, J. K. Christopher, S. Koenig, and F. Fioretto, "Multi-agent path finding in continuous spaces with projected diffusion models," *arXiv preprint arXiv:2412.17993*, 2024.

[27] J. Bradbury, R. Frostig, P. Hawkins, *et al.*, "JAX: Composable transformations of Python+NumPy programs," 2018. [Online]. Available: https://github.com/jax-ml/jax

[28] Z. I. Botev, D. P. Kroese, R. Y. Rubinstein, and P. L'Ecuyer, "The cross-entropy method for optimization," in *Handbook of Statistics*. Elsevier, 2013.

[29] H. Woo, K. R. I. Sanim, K. Okumura, G. Yang, A. Shankar, and A. Prorok, "Sanity: An agile brushless quadrotor for multi-agent experiments," 2025. [Online]. Available: https://openreview.net/forum?id=i5dKOANPZZ

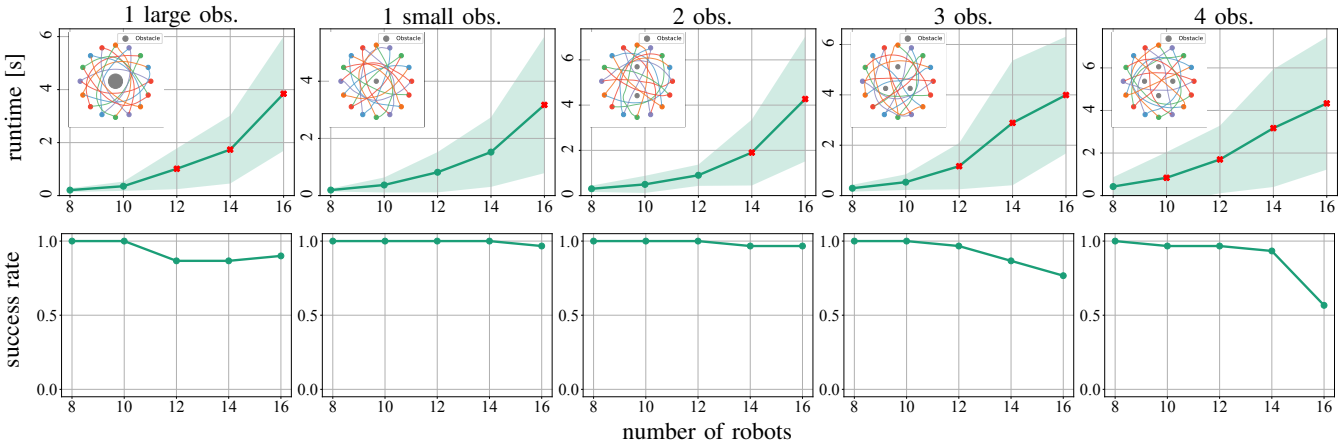


Fig. 4: Antipodal navigation scenarios for 16 robots with varying numbers of obstacles. Top row: The computation time for finding initial solutions is provided, averaged across 30 runs, excluding any failed runs. Bottom row: The success rate over 30 runs, with a maximum of 50 iterations.

APPENDIX

We now present some additional studies with D4ORM to showcase our ability to handle general target assignment scenarios (not only antipodal points on a circle/sphere), scaling up to more number of robots, and handling more complex workspaces that contain obstacles.

Generic Target Assignment. To validate that the success of our proposed method does not depend on exploiting biases or patterns in the problem, we test it on 30 random configurations with $\{8 \dots 16\}$ 2D holonomic robots. The environments have random initial and target positions, all generated within a square of size $D \times D$, with D as the diameter of the circle with antipodal points used in previous experiments. Fig. 5 (left) shows a solution instance for 16 robots. We also observe in Fig. 5 (right) that the method works reliably for all environments and is generally faster; the average time required to generate the first feasible solution for 16 robots is about 1 s, which is less than the circle with antipodal points scenario (≈ 2 s). This is because some of the trajectories do not interfere with each other due to geometric relationships in the random scenario, and are therefore often much easier to deconflict.

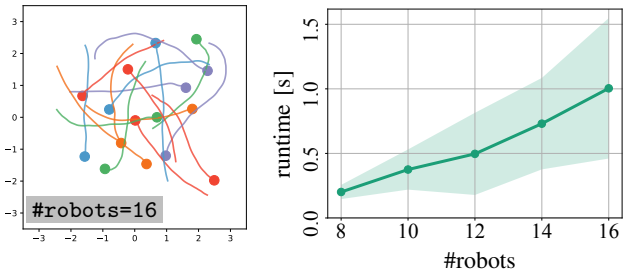


Fig. 5: A solution instance for a scenario with random targets, and time required by D4ORM to find initial feasible solutions.

Presence of Obstacles. We also tested our method in environments with obstacles, which can be easily addressed by a slight modification of the reward function of Eq. (3). In particular, for each state in the trajectories, we add a binary penalty term for whether the robot collides with any obstacle. Figure 4 shows that D4ORM is capable of handling obstacles. Meanwhile, as the workspace becomes more complex due to the addition of obstacles, the time required to obtain the first feasible solution increases, and D4ORM also encounters failures where a few robots are sacrificed for collision in order to achieve a high team reward. Dealing with obstacle-rich environments in a more systematic way (as opposed to a naïve binary penalty) is an interesting future direction.